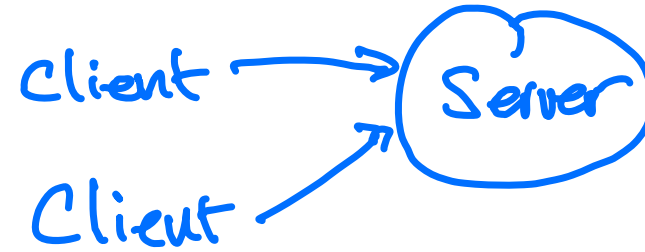


- RPC
- Faults
- Client/Server FS
 - NFS
 - AFS+ Coda
 - LBFS



Lecture 4: NFS

(Distrib. State + Evolution of v2-v4)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2026

Today's Topic: NFS

Focus on parts of NFS with State → Design Implications for scalability, semantics, and recovery

1.State: [The Role of Distributed State](#) [Download The Role of Distributed State](#)(skip later sections on Sprite)

John Ousterhout, 1991

2.NFSv2 Background: [Design and Implementation of the Sun Network Filesystem](#) (Focus on material before Implementation)

Sandberg, R., Goldberg, D., Kleiman, S., Walsh, D., and Lyon, B., Proceedings of the Summer 1985 USENIX Conference, Portland OR, June 1985, pp. 119-130.

3.NFSv3: [NFS Version 3: Design and Implementation](#)[Links to an external site.](#) (Focus on material before Section 5: Performance)

Brian Pawlowski and Chet Juszczak and Peter Staubach and Carl Smith and Diane Lebel and Dave Hitz, In USENIX Summer 1994

4.NFSv4: [The NFS Version 4 Protocol](#) [Links to an external site.](#)(Focus on material before Section 10: Attributes)

Brian Pawlowski and Spencer Shepler and Carl Beame and Brent Callaghan and Michael Eisler and David Noveck and David Robinson and Robert Thurlow, Proceedings of the 2nd international system administration and networking conference (SANE2000)

Why Today's Papers? Evolution of NFS protocol adding state to servers (continue stateful servers with AFS)

- Concepts shown in Practice: Many neat details about real systems and practical issues
 - Not SOSP/OSDI/Test of Time winner papers – more implementation details
- Stateless Servers
 - Server does not track which clients have files open()
 - Protocol must contain all info needed to complete requests
- Pros and Cons of Stateless Protocols
 - + Handles crashes easily
 - Confusing semantics for sharing with client caches
 - Poor performance (flush writes to persistent storage + overhead of getattr requests)
- Techniques for Stateful Servers:
 - Leases

Who is John Ousterhout?

Faculty member at UC Berkeley and Stanford University

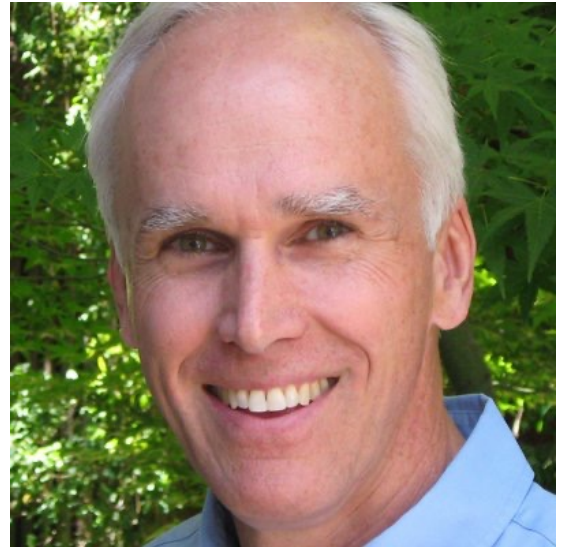
Tcl/Tk

Sprite Operating System

LFS (Log Structured File System)

Raft protocol

Very clear writer!

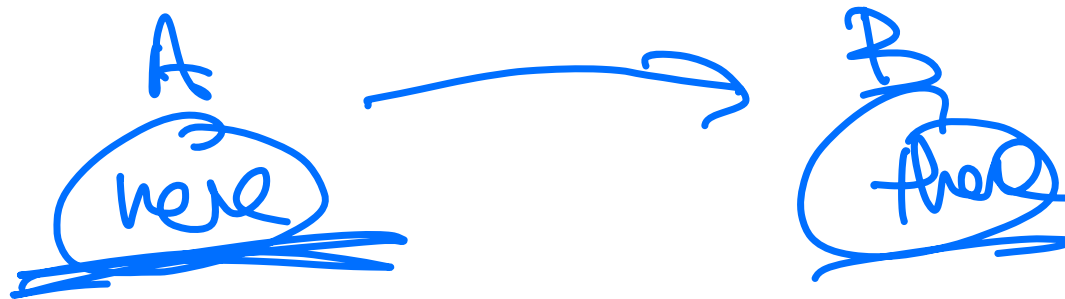


State and Distributed State

State? Includes all ~~describable~~ properties/values of a program + environment

Distributed State?

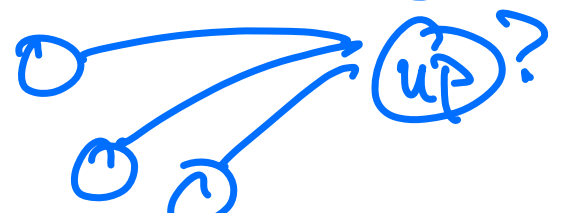
Info retained in one place that describes something somewhere else



Benefits of Distributed State?

1) Performance - local info, avail. immediately
caching!

2) Coherence - agree on same thing
- membership?



The diagram shows three small circles on the left, each with an arrow pointing to a larger circle on the right. The larger circle contains the text 'up?'.

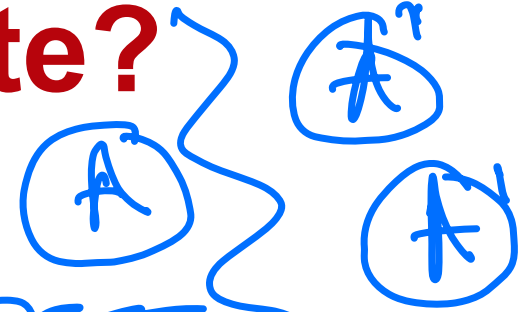
3) Reliability / Redundancy / Replication



The diagram illustrates data availability through replication. On the left, a circle containing the letter 'A' is crossed out with a large 'X'. In the center, a circle containing the letter 'A' has an arrow pointing to it from the text 'data still avail'. To the right, the word 'wrong' is written with an arrow pointing to it from the 'A' in the center. Further right, the letters 'A A' are written with an arrow pointing to them from the 'A' in the center.

Challenges of Distributed State?

1) Consistency - actually agree
- detect problem? use state data?
- prevent inconsistency - A read-only



2) Crash sensitivity

3) Time + space overheads

4) Complexity

NFSv2 Protocol (vs Implementation)

NFSv2 servers are stateless? *Yes*

Server cannot contain any essential

info in volatile storage

Can servers keep anything in RAM? *yes*

- non-essential

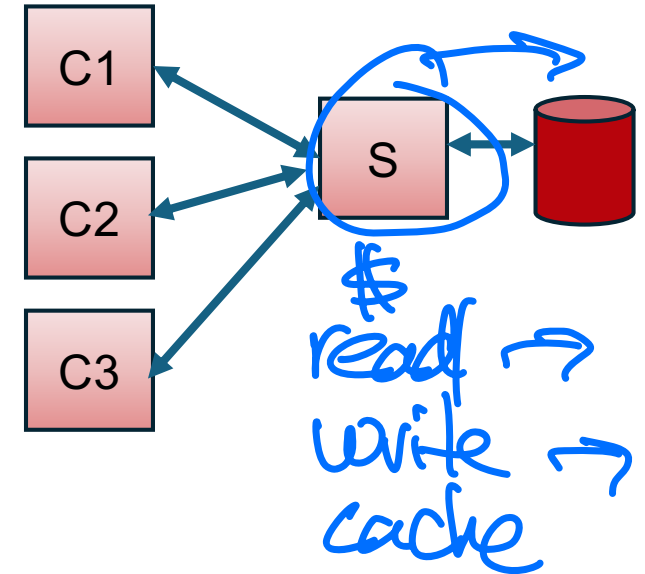
- cache, improve performance

Required property for NFSv2 requests?

every request must self-contained

- completely describe op

What state must clients maintain?



Idempotent Operations

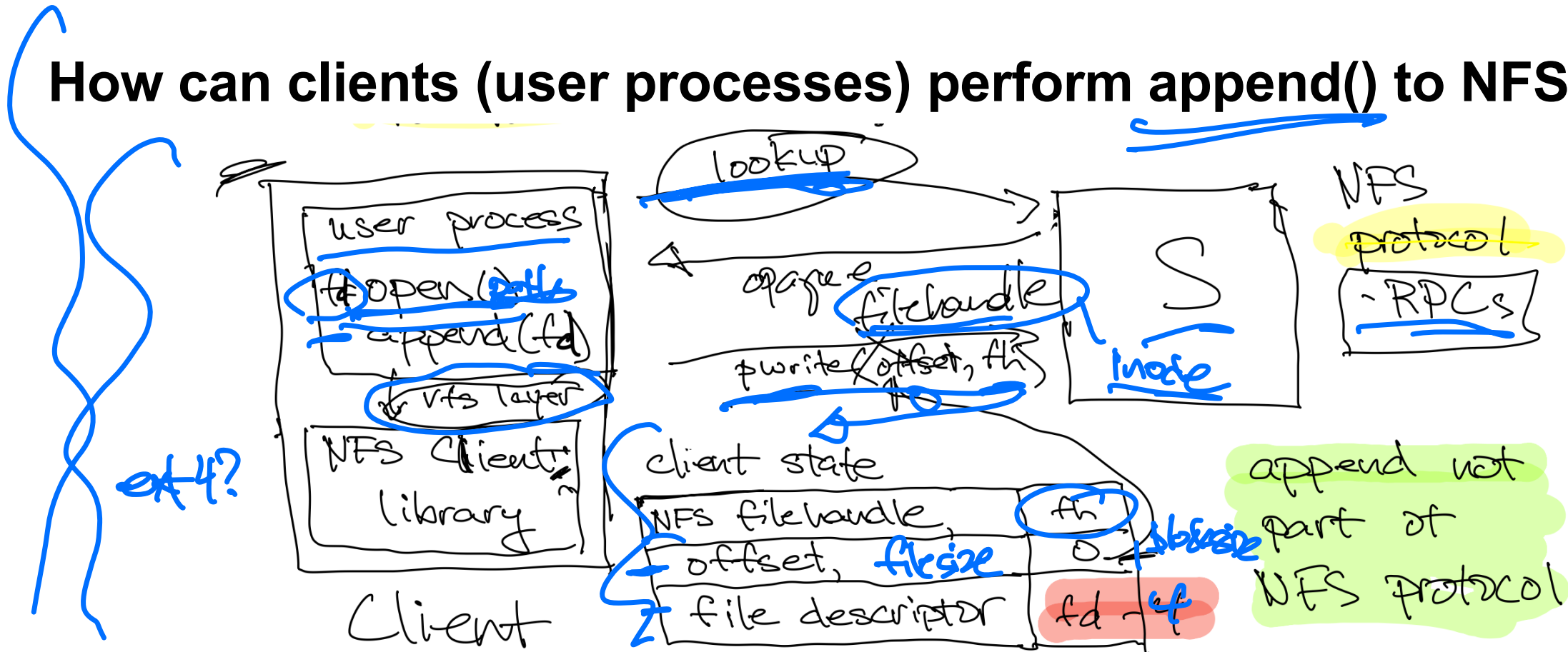
Idempotent:

$p=5$

$p \text{ read}(\underline{\text{offset}}, \text{file})$ $p \text{ write}(\underline{\text{offset}}, \text{file})$

Can be executed multiple times with same results as only executed once

How can clients (user processes) perform append() to NFSv2 file?



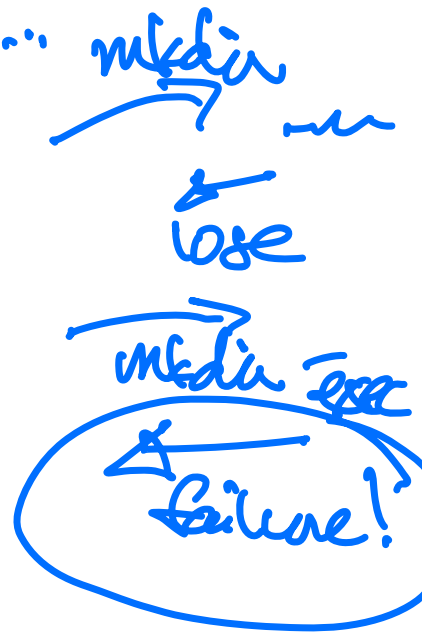
Are all NFSv2 ops idempotent?

Example of non-idempotent op? *not completely idempotent...*

create (mkdir - mkdir - created)

1 - success: dir didn't exist before, created!

0 - failure: dir already existed, didn't create!



How did v2 handle non-idempotent ops?

- Reply cache, remember return code,
- See ~~etc~~ id, return code

What if server crashes?

- lose reply cache, return confusing result
- server remembers nothing!

Stateless NFSv2 Server: Advantages

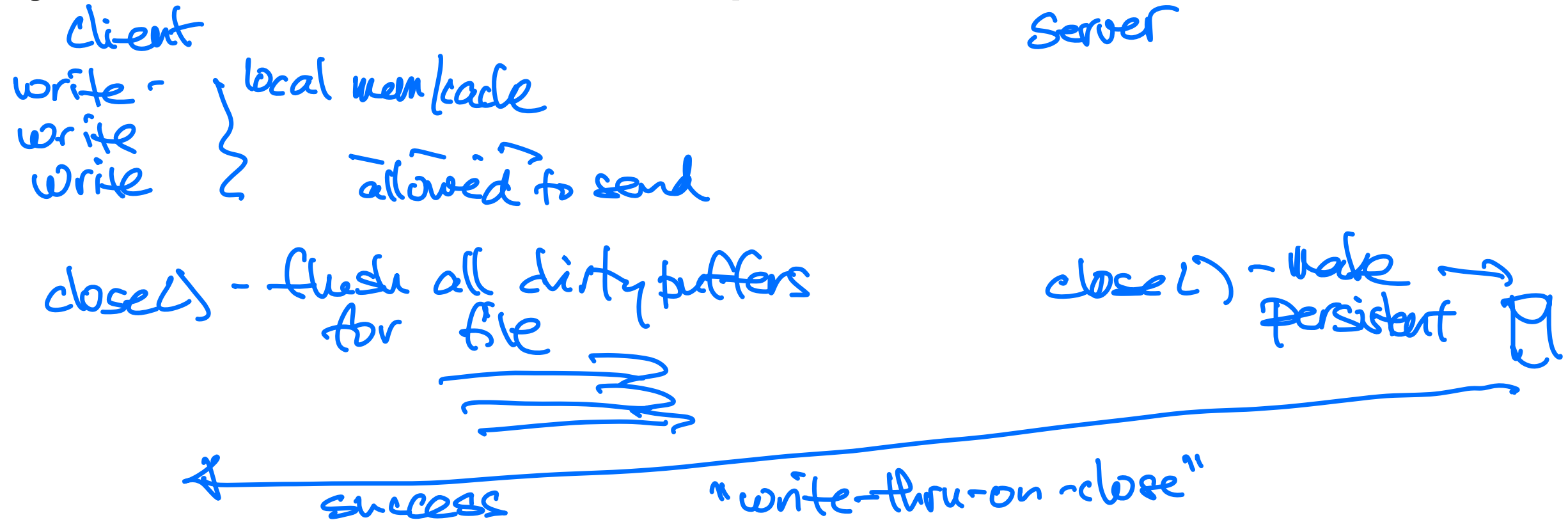
Advantages of stateless NFSv2 Server?

What does client do if server crashes?

What does server do if client crashes?

Stateless NFSv2 Server: Disadvantages

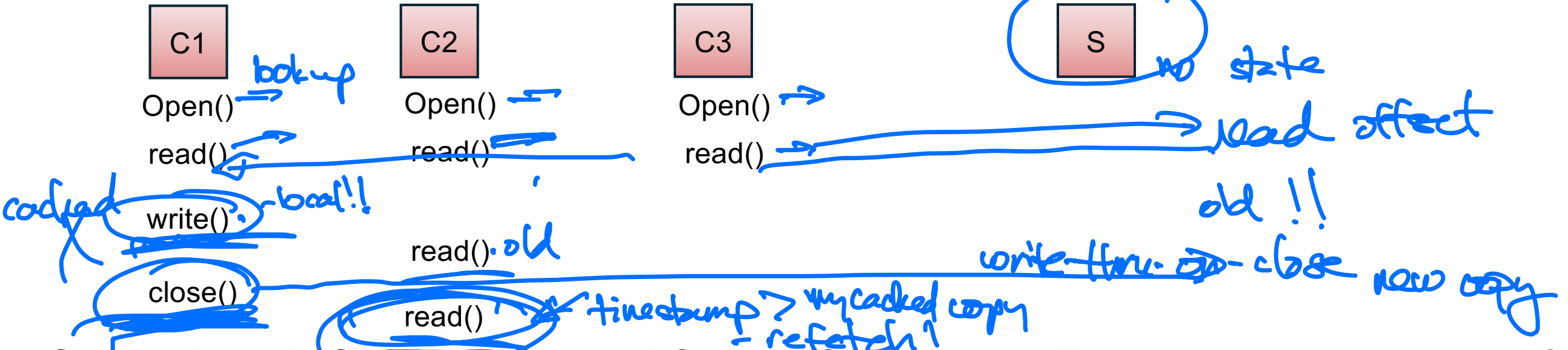
Why do stateless v2 servers hurt performance of writes?



How to fix performance? (within NFSv2 protocol?)

→ NVRAM

NFSv2 Client Caching: Consistency?



Client Block Cache: Needed for performance! Fundamental problem?

- don't know about shared state

How can clients discover data is stale?

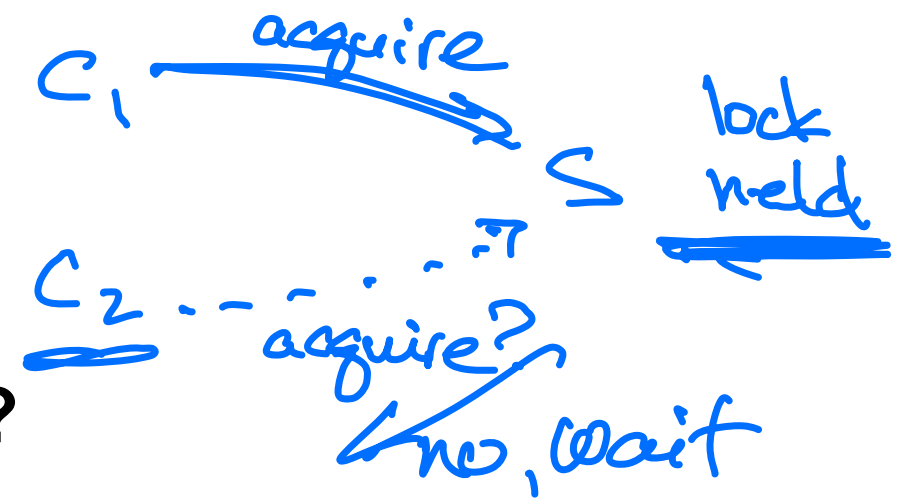
getattr → server, timestamps - written
- assume good for 3 secs, getattr

Performance problems? - getattr on server

Rules for checking for changes before read()? next open()?

Distributed Locks

Can stateless server provide locks for files?



Solution? in NFSv2

- part of another service!
- lock server

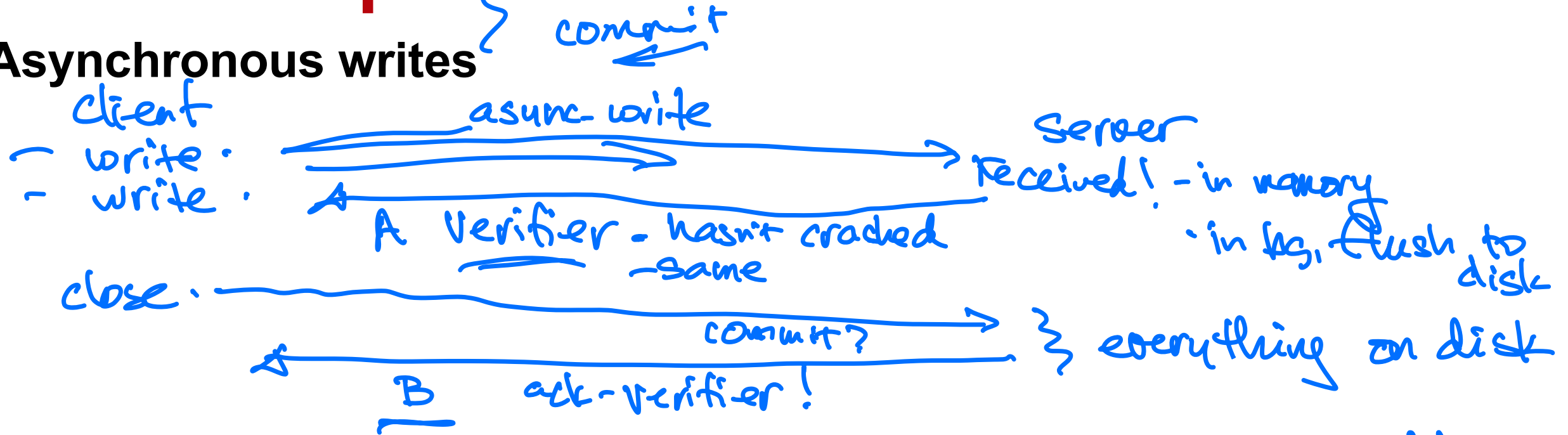
NFSv3: Addresses 3 problems

- X 1) 64 bit file size support
- 2) Perf problems w/ writes
- 3) Too many `getattr()`

NFSv3 Stateless Servers? **Why?** *Yes* *Know how!*

NFSv3: Improves Write Performance

Asynchronous writes



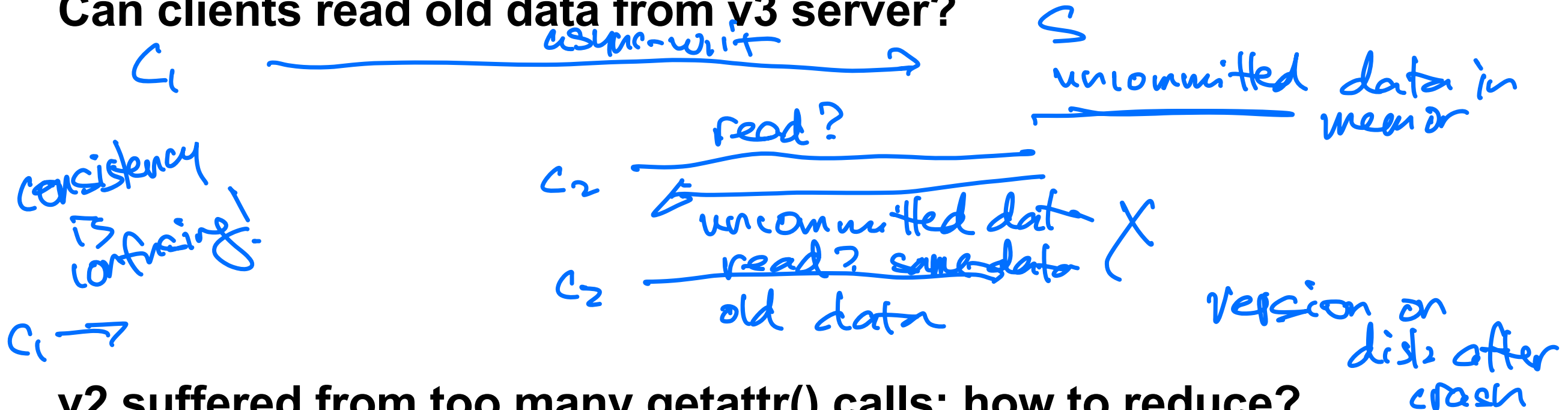
What must client do if server crashes? - tell, verifier changed!
- client has to remember + redo all prev. writes (not ack)
- can ask for early commit if client can't buffer all prev writes

How will client know that server crashed?

- verifier changes on reboot (tiny bit of state!)

NFSv3: Uncommitted Data can be Lost

Can clients read old data from v3 server?

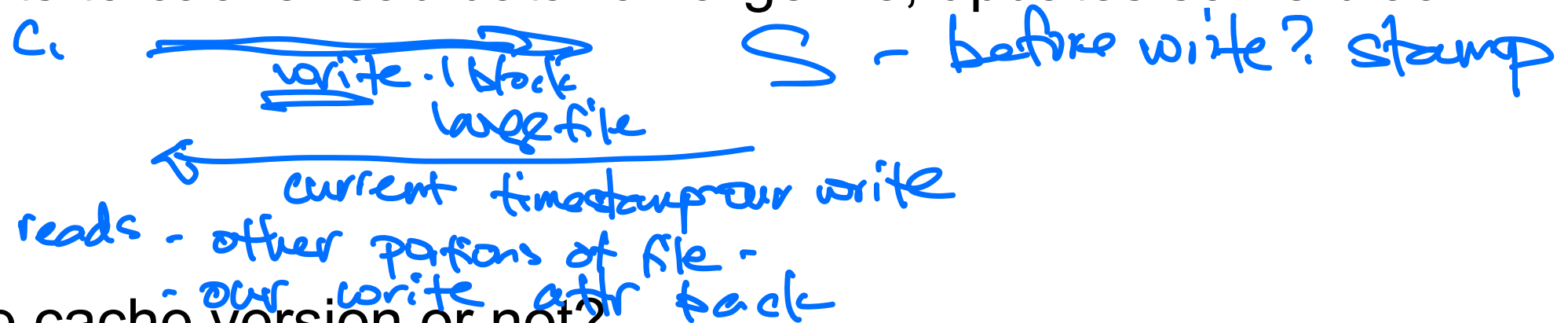


v2 suffered from too many `getattr()` calls; how to reduce?

- Engineering
 - any call - returns attr - per file!
 - all files in dir

NFSv3: When are getattr results useless?

Client wants to cache read data for large file; updates some block...



Read: Use cache version or not?

- 1) assume no one else wrote - could read old data
- 2) throw away cached data - perf. worse

Solution? - pre-op: timestamp before
- post-op:

NFSv4: Stateful Servers

What is a COMPOUND procedure? Why introduced?

put fh
lookup
get fh
gettr
read

- reduce network roundtrips
- improve latency

Semantics? - not a transaction
- interleaving w/ other RPCs

Complexities?

- failure in middle

NFSv4: Introduces open() + close()

Why?

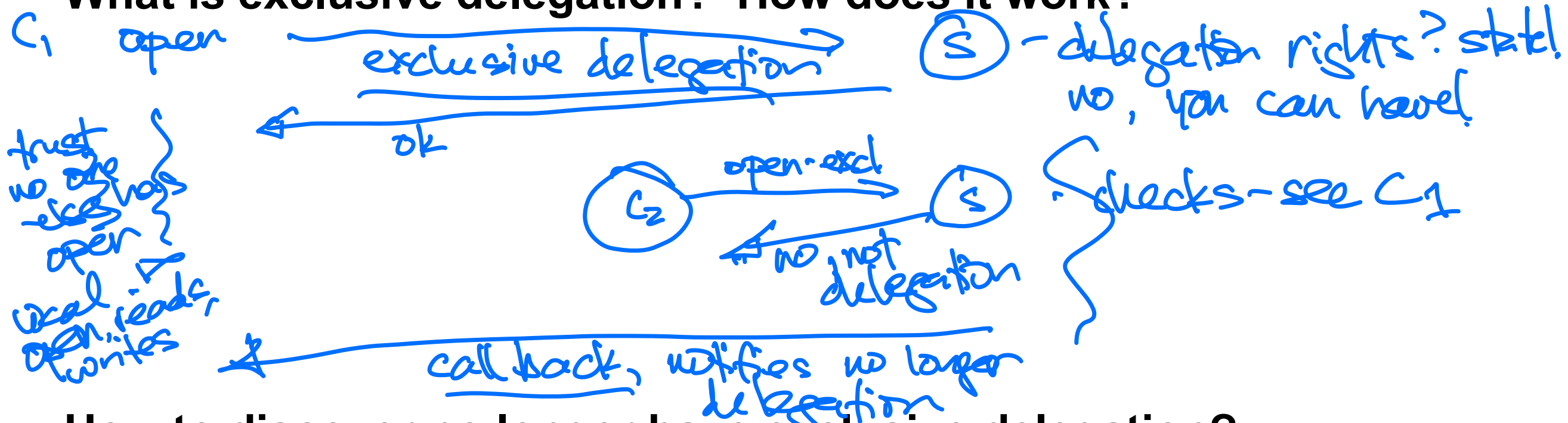
- File locking - Windows required
- Improve perf.
- Better consistency - ad hoc ^{still}

When can future open()'s be kept local on client?

open w/ delegation $\leftarrow$ exclusive
read-only

NFSv4: Delegation of Open()

What is exclusive delegation? How does it work?



How to discover no longer have exclusive delegation?

- 1) call back
- 2) lease expiration - can't reach ^{due to network}

What are advantages of delegation?

- avoid `getattr`, keep all ops local

NFSv4: Crashes with Delegation

Delegation is **PERFORMANCE** optimization;

Delegation recovery is opportunistic: performance depends on it, not correctness

Assume **client c1** with ex. delegation crashes;

How does server delegate to different client, c3?

c1 assume deleg. for lease interval ...
Server wait for lease to expire
safe to give to c3

Assume **server** crashes when c1 had delegation;

How does server delegate to c3? What state when server reboots?

lease 


reboots - wait lease interval
give to c3

NFSv4: Consistency

What is guaranteed if two clients have file open for read/write?

w/o locks, same ad hoc as v2

Share reservation (Locks):

Client can open file and deny other clients open access to same file

Delegations vs. Locks

Aspect	Locks	Delegations
<u>Purpose</u>	<u>Correctness</u> <i>exclus access</i>	<u>Performance</u>
Required for safety?	Yes	No
State loss impact	Data corruption risk	Performance loss
Must be reclaimed on server crash?	<u>Yes</u>	No
<u>Server can refuse reclaim?</u>	<u>No (during grace)</u>	Yes
Typical persistence	<u>In-memory</u>	In-memory

NFSv4: How to detect crashes?

Server must know if **client** crashed for correct recovery

Server tracks:

Client ID (chosen by server), Verifier ID (chosen by client), ...state (locks)...

Client reboots:

New Verifier ID (based on timestamp)

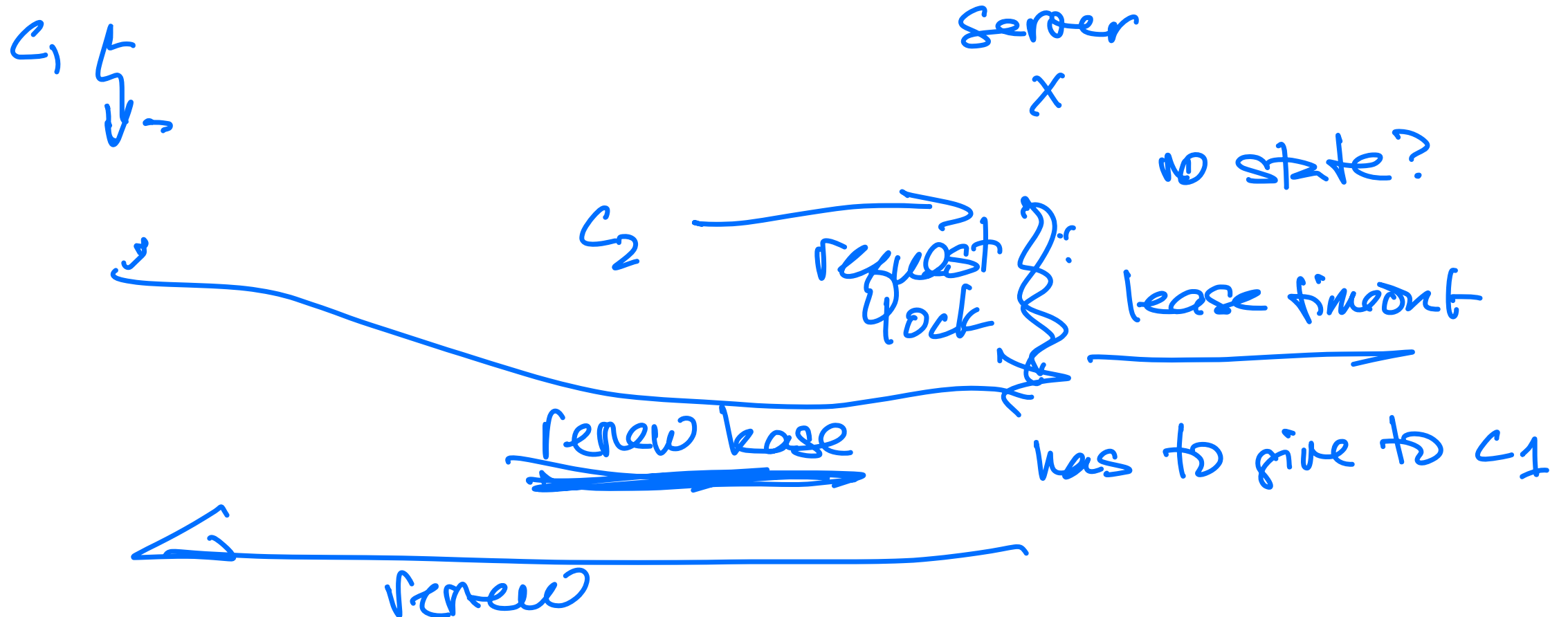
detect mismatch

- client crashed → release all locks!!!
- lease expires, release locks

NFSv4: How to detect crashes?

Client must know if server crashed for correct recovery

What if server crashes? Assume c1 had lock...



Summary

NFS v2 + v3: Known for stateless servers

- Clients still need state (track open files); Servers can still cache data for performance
- Open(), close() not part of NFS protocol
- Use polling of getattr() to discover if file has changed
- Poor performance
- Poor consistency

NFSv3: Practical optimizations to improve performance

- Asynchronous writes with commits
- Improved getattr

NFSv4: State on server

- Tracks clients with file open
- Delegation: Keep ops local for performance
 - Reverts to getattr() if sharing
- Locks: Used to guarantee exclusive access for correctness
- Leases help handle crashes
- Different sharing semantics than AFS

Announcements

Reading Group 1: Continue for Week 2 due Sunday

- Same group as Week 1

Mini-Project 1: RPC for Client/Server KV Store –

- Groups picked or assigned
- Sign up for Cloudlab account
- TA Office Hours
- Grade with quick demo with us

Friday: AFS and Coda

•**AFS Background:** [Scale and Performance in a Distributed File System](#)

Howard, J.H., Kazar, M.L., Menees, S.G., Nichols, D.A., Satyanarayanan, M., Sidebotham, R.N., and West, M.J. ACM Transactions on Computer Systems, Vol. 6, No. 1, February 1988, pp. 51-81.

SIGOPS Hall of Fame Award

•**Coda:** [Disconnected Operation in the Coda File System](#)[Links to an external site.](#)

James J. Kistler, M. Satyanarayanan, 13th Symposium on Operating Systems Principles, Asilomar, California, pp. 213-225. October 1991. **SIGOPS Hall of Fame Award**